

LAB 4 - VIBECODING

Treasure Hunt - Chat / LLM History

Name: Priyanka B M

SRN: PES1UG24AM207

LLM used: ChatGPT

Repository: priyankabm11/PES1UG24AM207_SE

About this document

This is my ChatGPT working history for Lab 4. I used ChatGPT to understand the Treasure Hunt README, inspect the existing game, add the four required features one by one, test them, and complete the GitHub work. I have kept the explanations short and in the order in which I completed the work.

1. Starting the project

I cloned the assigned Treasure Hunt repository, inspected the README, and ran the original Pygame game before making changes. I later copied the updated game into my personal lab repository under the Lab-4 folder.

My prompt:

I have this VibeCoding Treasure Hunt lab. Tell me what I need to do and guide me step by step.

ChatGPT guidance:

The main steps were to read the README, run the original game, record the BEFORE video, implement Tasks 1-4 one by one, test the changes, record the AFTER video, and push the work to my personal GitHub repository.

What I did: The original game ran successfully and I recorded my BEFORE video before making the feature changes.

2. Task 1 - Floor Traps

My prompt:

Implement ONLY Task 1: Floor Traps. Add floor traps that send the player back to the starting position when stepped on, with a clear status message, while preserving the existing key and chest functionality.

ChatGPT guidance:

The starting position should be stored explicitly when the game is reset. Trap tiles should be placed on normal floor tiles away from the starting room, key, and chest. When the player steps on a trap, the player should be returned to the exact starting position and a status message should be shown.

Code used:

```
TRAP = 4

if cell == TRAP:
    self.player.rect.topleft = self.start_pos
    self.status = "Trap! Back to start!"
```

Testing: I ran the game, checked trap behaviour, and confirmed that the player was reset to the starting position with a clear message.

Commit message: Add Lab 4 treasure hunt

3. Task 2 - Enemy Guard

My prompt:

Implement ONLY Task 2: Enemy Guard. The guard should patrol back and forth near the chest, and touching it should reset the player to the start.

ChatGPT guidance:

The guard must use two fixed patrol points rather than choosing random directions. It should move continuously toward one endpoint, reverse direction at the endpoint, and continue patrolling. A collision with the player should reset the player to the saved starting position and show a status message.

Code used:

```
class Guard:
    def __init__(self, x, y, patrol_start, patrol_end):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.patrol_start = patrol_start
        self.patrol_end = patrol_end
        self.direction = 1

    def move(self, grid, rows, cols):
        target = self.patrol_end if self.direction == 1 else self.patrol_start
        # move toward target and reverse at the endpoint
```

Additional change: I fixed the guard behaviour after observing that a randomly moving guard could block the only corridor to the key. The corrected guard patrols between two usable points near the final area instead of wandering randomly.

Testing: I ran the game and checked that the guard was visible, patrolled continuously, and reset the player when a collision occurred.

4. Task 3 - Mini-map

My prompt:

Implement ONLY Task 3: Mini-map. Show the dungeon layout in a corner, distinguish walls from floor, and display the player position in real time.

ChatGPT guidance:

The mini-map should use the existing grid instead of a separate hardcoded map. Walls and walkable cells are drawn at a smaller scale, and the player position is calculated from the player rectangle each frame.

Code used:

```
map_w = 180
map_h = 135
map_x = WIDTH - map_w - 10
map_y = 10

if cell == WALL:
    color = (45, 40, 55)
else:
    color = (190, 180, 160)

player_c = self.player.rect.centerx // TILE
player_r = self.player.rect.centery // TILE
```

Testing: The mini-map appeared in the top-right corner, reflected the dungeon layout, and the blue player marker moved with the player.

5. Task 4 - Inventory UI

My prompt:

Implement ONLY Task 4: Inventory UI. Add a HUD inventory slot that is empty at the start and displays a key icon immediately after the key is collected. The inventory should reset when the game is restarted.

ChatGPT guidance:

The existing `Player.has_key` state already represents whether the player has collected the key. The HUD can use that Boolean to render either an empty slot or a simple key icon, so the visual state updates immediately after pickup and clears automatically on restart.

Code used:

```
inventory_x = 8
inventory_y = ROWS * TILE + 8
inventory_size = 34

pygame.draw.rect(self.screen, (70, 70, 85),
(inventory_x, inventory_y, inventory_size, inventory_size))

if self.player.has_key:
    pygame.draw.circle(self.screen, (255, 240, 60),
(inventory_x + 12, inventory_y + 17), 6, 2)
```

Testing: I checked the empty slot at the beginning, collected the key, and verified that the yellow key icon appeared immediately in the inventory slot. Restarting the game resets the inventory because `Player.has_key` is initialized to `False` again.

Commit message: Add inventory UI

6. GitHub / Version Control Work

For the lab, I pushed the updated code and videos to my personal GitHub repository. I did not push changes directly to the SETAPESU26 repository or create a pull request to the main repository.

```
Relevant commands:

git add Lab-4/game.py
git commit -m "Add mini-map"
git push origin main

git add Lab-4/game.py
git commit -m "Add inventory UI"
git push origin main

git add Lab-4/before.mov Lab-4/after.mov
git commit -m "Add before and after videos"
git push origin main
```

I also removed macOS `.DS_Store` files and verified that the final working tree was clean and up to date with `origin/main`.

7. Final Testing

After completing the changes, I tested the required behaviours from the README.

Feature	What I checked
Traps	Player is reset to the start and a clear trap message appears.
Guard	Guard patrols between fixed points and collision resets the player.
Mini-map	Shows the dungeon layout, distinguishes walls/floor, and tracks the player.
Inventory	Empty initially and shows the key icon immediately after pickup.
Chest	Key can still be used to reach the treasure.
Restart	New run resets player state and inventory.

8. Final Video

BEFORE video: before.mov

AFTER video: after.mov

I recorded the BEFORE video before making the feature changes and the AFTER video after completing and testing the required tasks. The AFTER video shows the updated game with the mini-map and inventory UI working.

9. Submission

The final Lab-4 folder contains the updated game code, the BEFORE and AFTER videos, and this ChatGPT working-history PDF. The README also asks for the Chat/LLM page link containing the complete chat history.